ESTRUTURA DE DADOS | ÁRVORES BINÁRIAS DE BUSCA

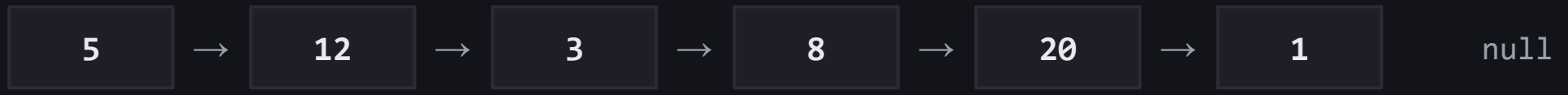
Árvores Binárias de Busca

A estrutura que equilibra busca, inserção e remoção

IFSP Câmpus Campos do Jordão | Tecnologia em ADS | Prof. Geterson Policarpo Nunes

Lista encadeada

A lista encadeada é boa para inserção e remoção, basta ajustar ponteiros.
Mas buscar um elemento exige percorrer do início ao fim.



Inserção e remoção

O(1)

Ajusta ponteiros, rápido em qualquer tamanho.

Busca

O(n)

Percorre do início ao fim, lento em listas grandes.

Ordenação

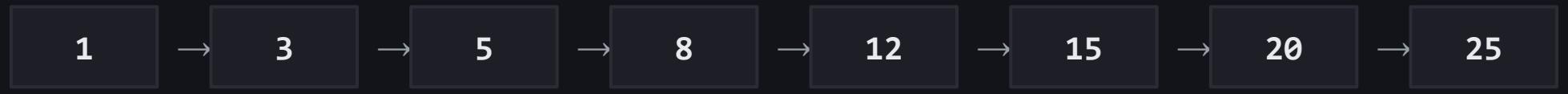
O(n log n)

Não mantém ordem, ordenar exige algoritmo extra.

Busca binária

A busca binária é rápida, divide o espaço de busca pela metade a cada passo.

Mas exige que o vetor esteja sempre ordenado.



Buscando 12: compara com 8, vai para a direita, encontra na segunda comparação.

Busca

O(log n)

Divide pela metade a cada passo.

Manter ordenado

O(n log n)

Ordenar o vetor custa mais que buscar nele.

Inserção e remoção

O(n)

Mover elementos para manter a ordem é caro.

Tabela hash

A tabela hash é excelente para busca, a posição é calculada e não percorrida.
Mas ela não mantém ordem.

0	1	2	3	4	5	6	7
5	12		3, 8		20	1	

Busca

O(1) em média

Acesso direto pela chave.

Inserção e remoção

O(1) em média

Colisões e rehashing custam mais.

Ordem

Não mantém

Percorrer em ordem exige ordenar depois.

No pior caso, com muitas colisões, a busca degrada para O(n).

Onde a árvore entra

Nenhuma estrutura vista até aqui resolve os três casos ao mesmo tempo.

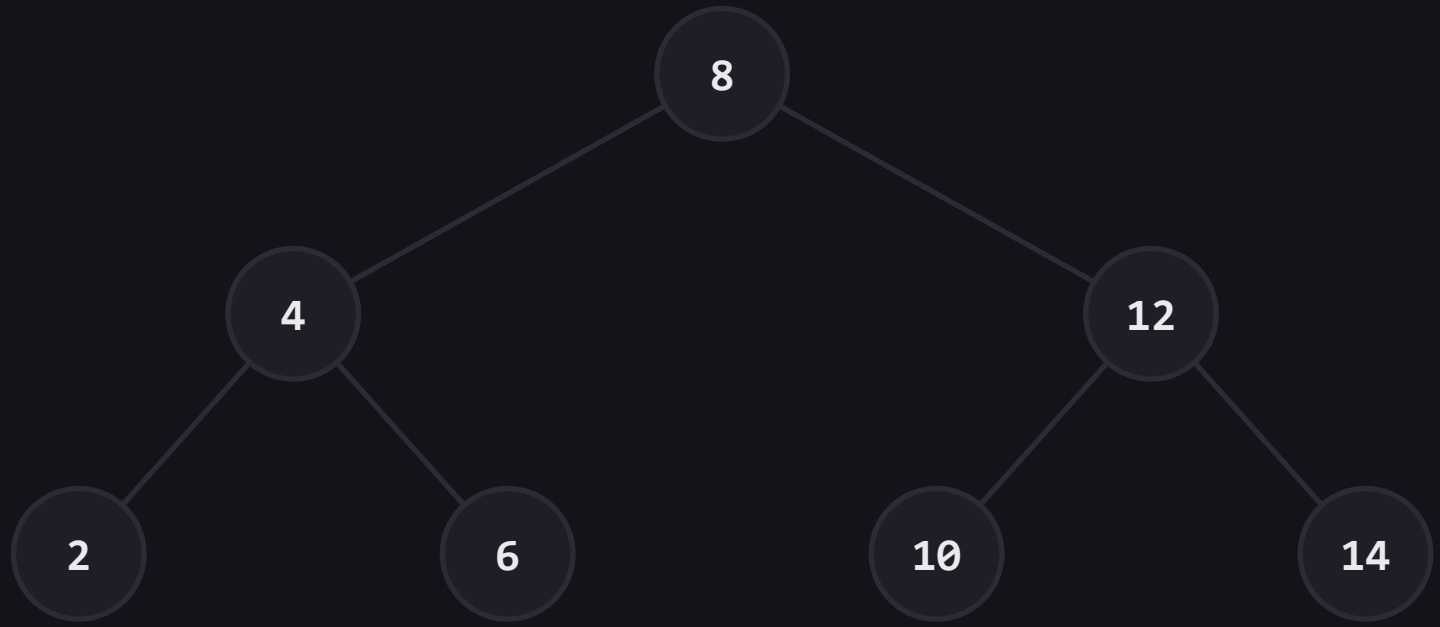
Estrutura	Busca	Inserção	Remoção	Ordenado
Lista encadeada	O(n)	O(1)	O(1)	não
Vetor ordenado	O(log n)	O(n)	O(n)	sim
Tabela hash	O(1)	O(1)	O(1)	não
Árvore binária de busca	**O(log n)**	**O(log n)**	**O(log n)**	**sim**

A árvore binária de busca equilibra os três casos e ainda mantém os dados em ordem.

O que é uma árvore

Uma estrutura hierárquica formada por nós ligados por arestas.

Cada nó pode ter filhos, mas cada filho tem um único pai.



Em uma árvore binária, cada nó tem no máximo dois filhos.

Raiz, nó e folha

Raiz

O único nó sem pai.

É o ponto de entrada da árvore, toda travessia começa por ela.

Toda árvore tem exatamente uma raiz.

Nó

Cada elemento da árvore.

Guarda um valor e as ligações para os filhos.

Em uma árvore binária tem zero, um ou dois filhos.

Folha

Nó sem filhos.

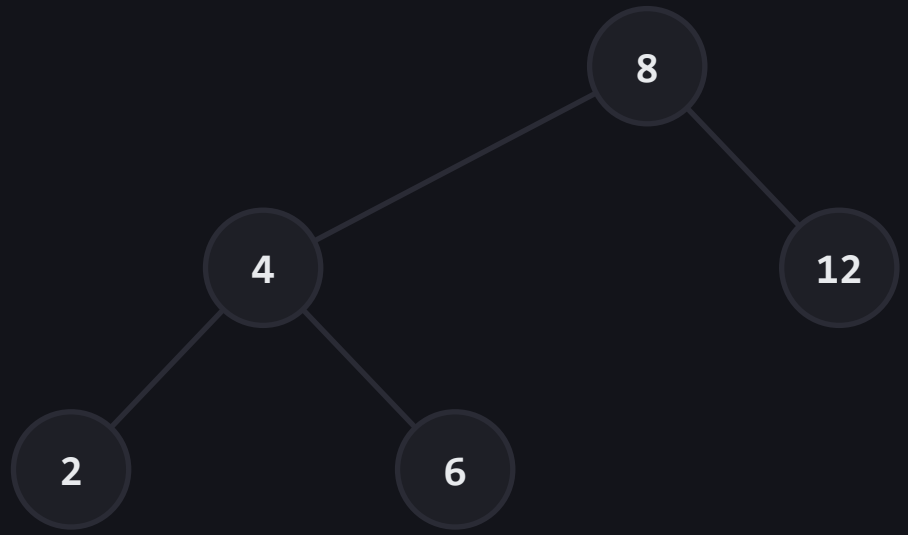
Os dois ponteiros apontam para nullptr.

Está sempre no nível mais externo da árvore.

A propriedade que define a BST

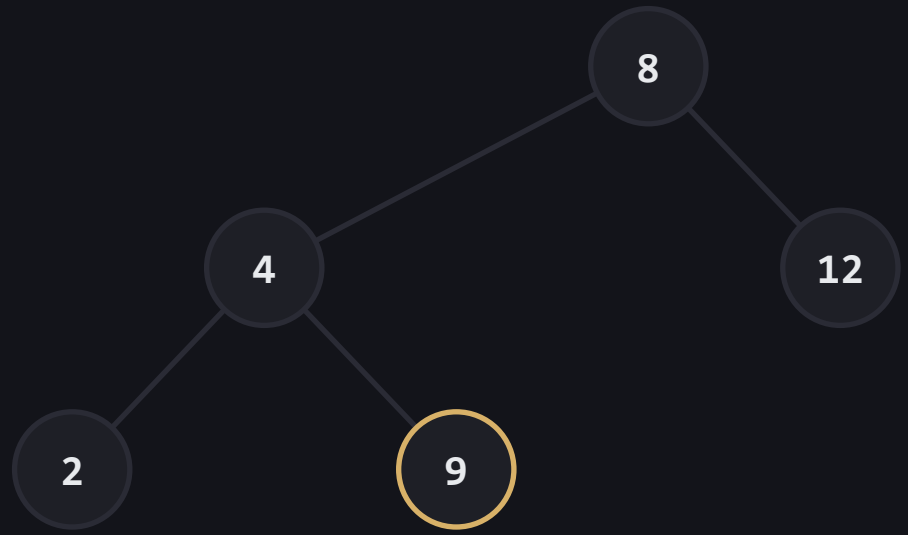
Para todo nó: a subárvore da esquerda tem apenas valores menores que ele, e a subárvore da direita tem apenas valores maiores.

BST válida



2, 4 e 6 são todos menores que 8.

BST inválida



9 é maior que 8, mas está na subárvore esquerda.

Sem essa propriedade a estrutura continua sendo uma árvore binária, mas a busca eficiente deixa de funcionar.

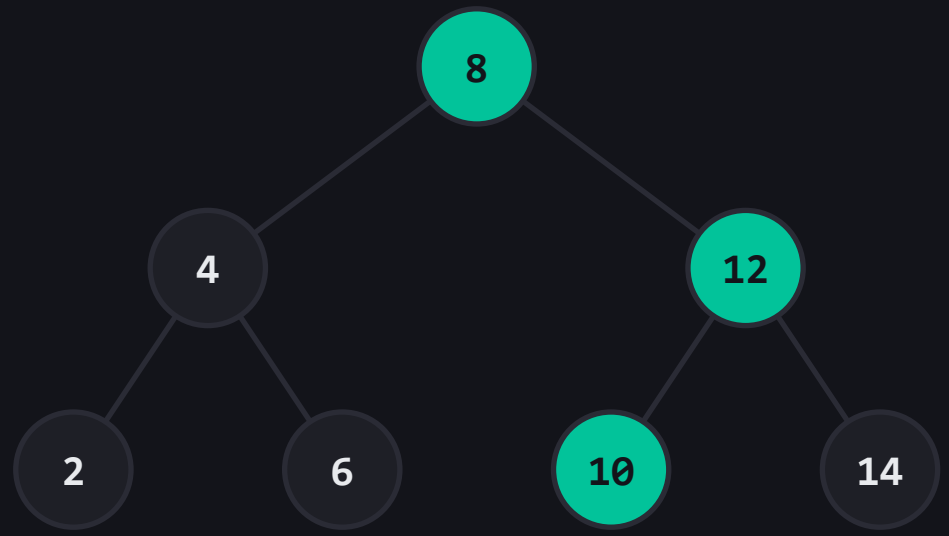
Como encontrar um valor

A cada nó decidimos ir para a esquerda ou para a direita.

Nunca é preciso olhar os dois lados.

Buscando o valor 10

1. Em 8: 10 é maior, vai à direita.
2. Em 12: 10 é menor, vai à esquerda.
3. Em 10: encontrado.

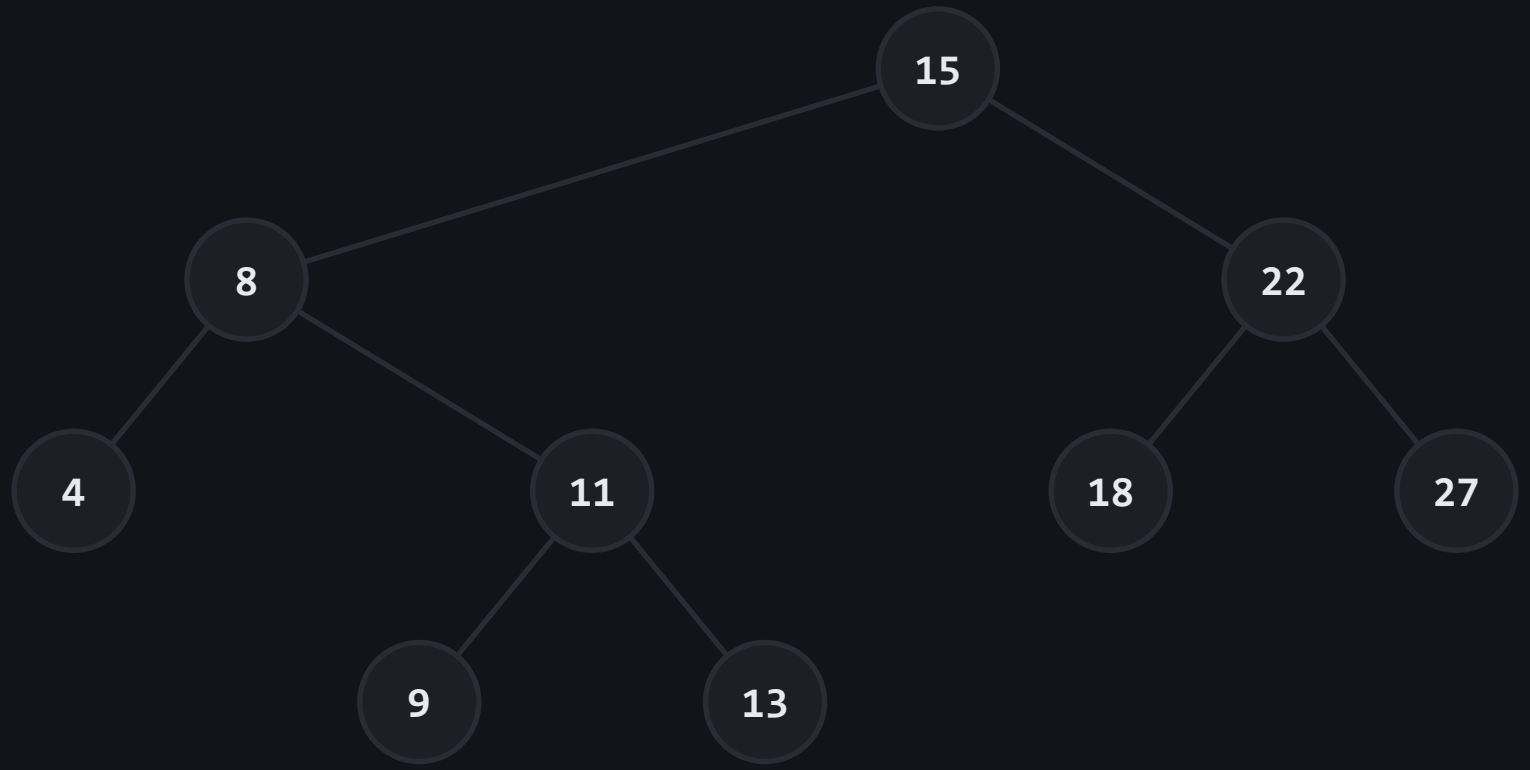


Se o valor é igual ao nó, encontrou. Se é menor, busca na esquerda. Se é maior, busca na direita.

PARTE 3 | EXERCÍCIO EM AULA

Rastrear a busca no papel

Para cada valor abaixo, percorra a árvore e anote os nós visitados até encontrar o valor ou concluir que ele não existe.



Busque

13

22

9

20

4

Para cada busca: quais nós foram visitados e quantas comparações foram necessárias?

Rastrear a busca no papel

Valor	Nós visitados	Comparações
13	15, 8, 11, 13	4
22	15, 22	2
9	15, 8, 11, 9	4
20	15, 22, 18	3, não existe
4	15, 8, 4	3

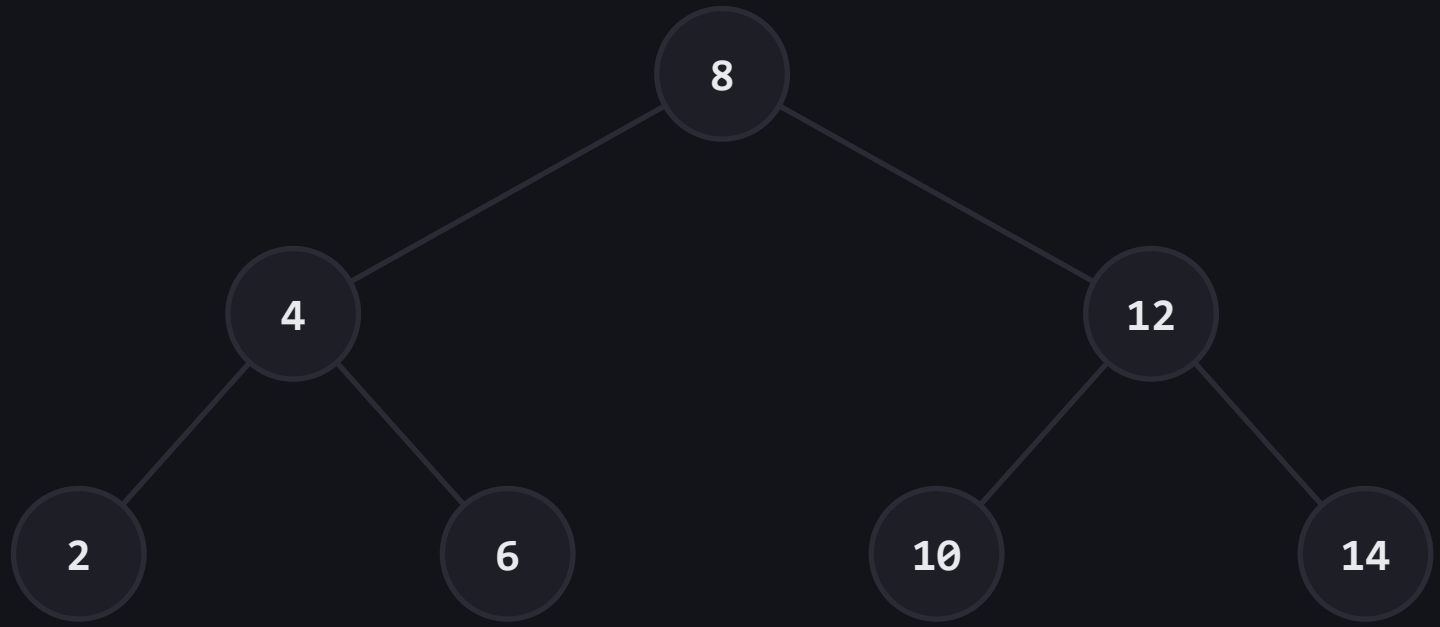
A busca do 20 termina em 18: o valor seria maior, mas a direita de 18 está vazia.

Nove nós e nunca mais de quatro comparações. Uma lista de nove exigiria até nove.

Onde entra o valor 7?

A árvore abaixo respeita a propriedade da BST.

Existe um único lugar onde o 7 pode entrar sem quebrar essa propriedade.



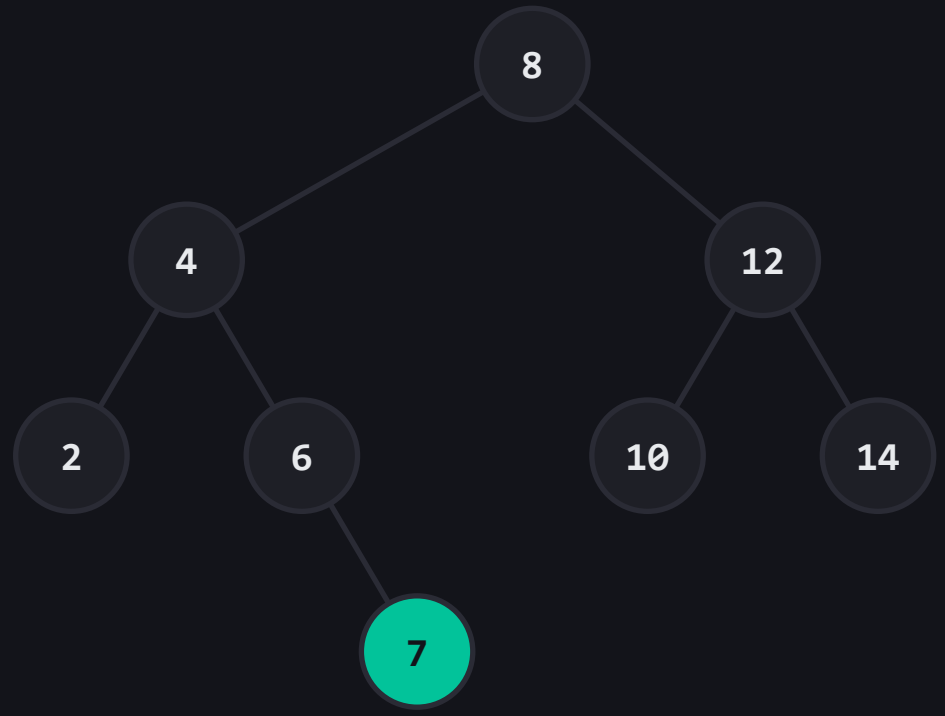
Qual é esse lugar, e que caminho percorremos até chegar nele?

Como adicionar um valor

Inserir é igual a buscar. Seguimos o caminho até encontrar uma posição vazia, e é ali que o novo nó se encaixa.

Inserindo o valor 7

1. 7 é menor que 8, vai à esquerda.
2. 7 é maior que 4, vai à direita.
3. 7 é maior que 6, vai à direita.
4. A posição está vazia, insere aqui.

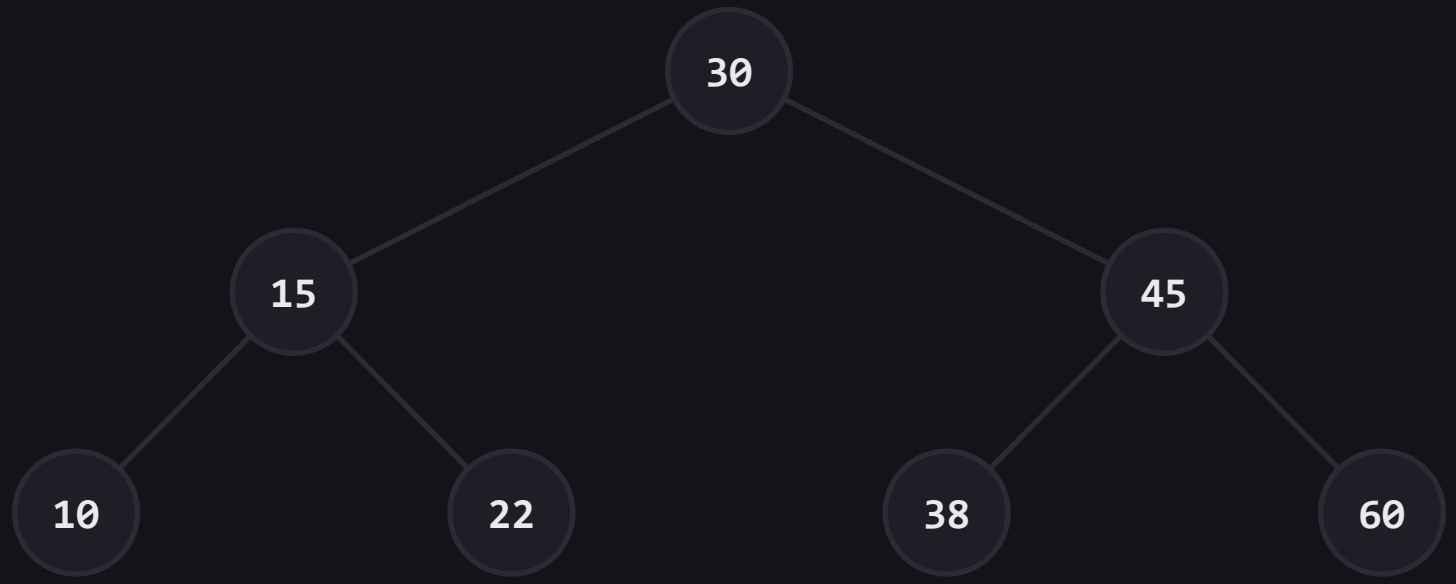


O novo nó entra sempre como folha. Nenhum outro nó precisa mudar de lugar.

Inserir em uma árvore existente

Copie a árvore no papel e insira os valores na ordem indicada.

Desenhe o estado da árvore depois de cada inserção.



Insira

27

5

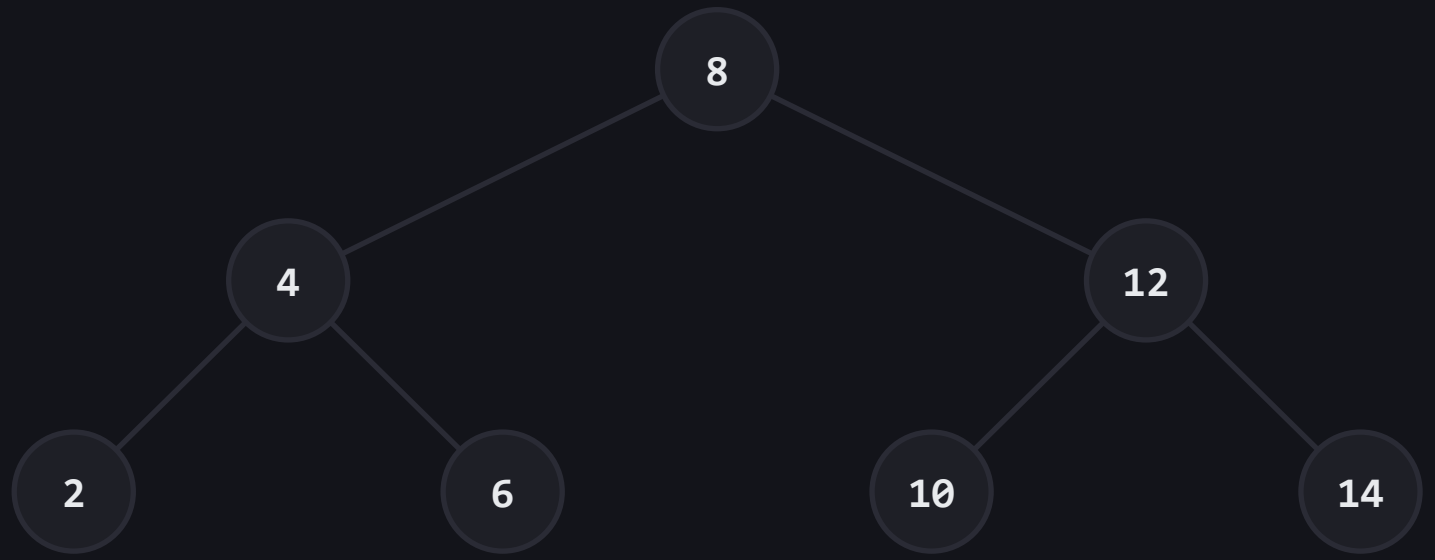
50

33

Para cada valor, anote também os nós comparados no caminho até a posição vazia.

Em-ordem: esquerda, raiz, direita

Visita a subárvore esquerda, depois o nó atual, depois a subárvore direita.



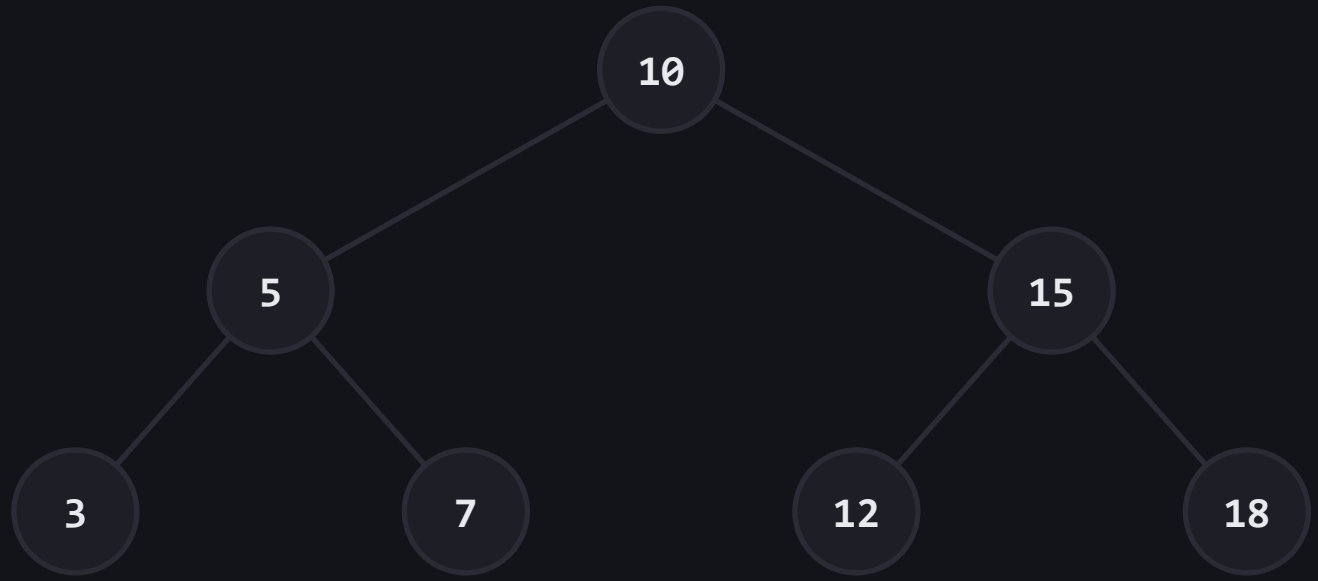
Ordem de visita

2 → 4 → 6 → 8 → 10 → 12 → 14

Em uma BST, o percurso em-ordem sempre produz os valores em ordem crescente.

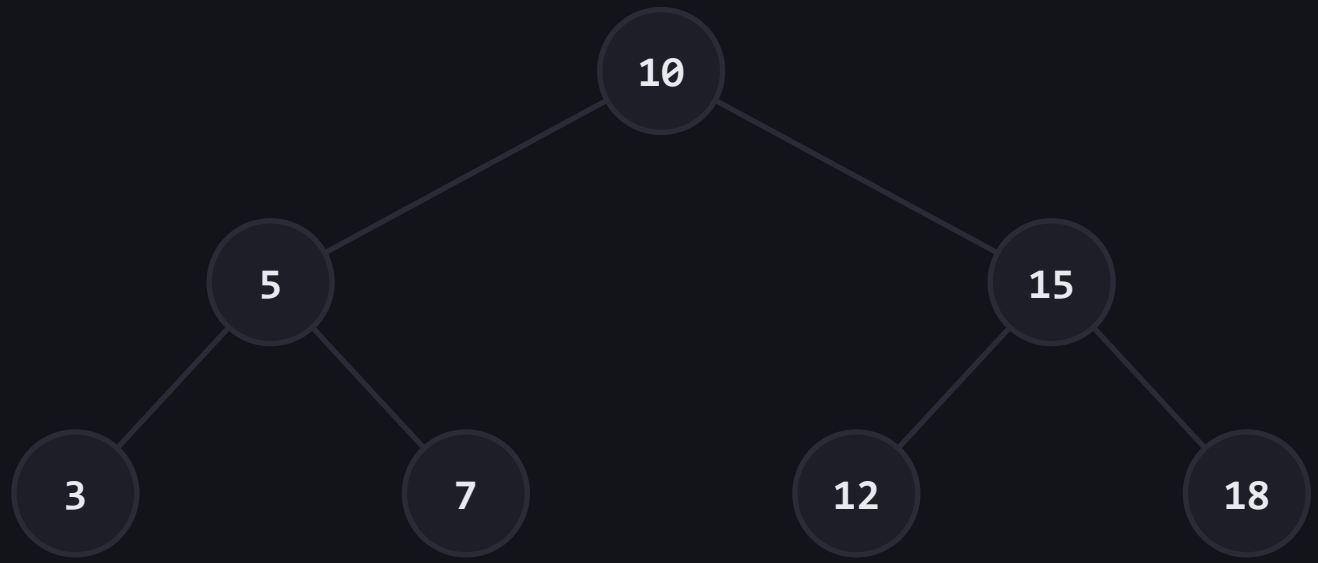
Em-ordem

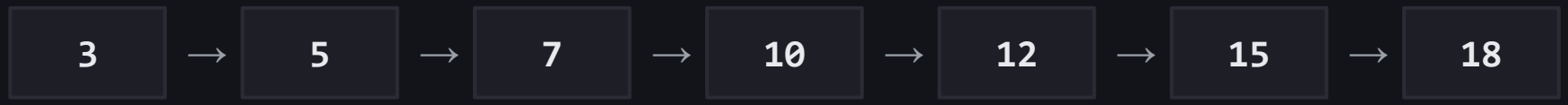
Escreva a sequência de visita do percurso em-ordem para a árvore abaixo.



Em-ordem: ____ , ____ , ____ , ____ , ____ , ____ , ____

Em-ordem





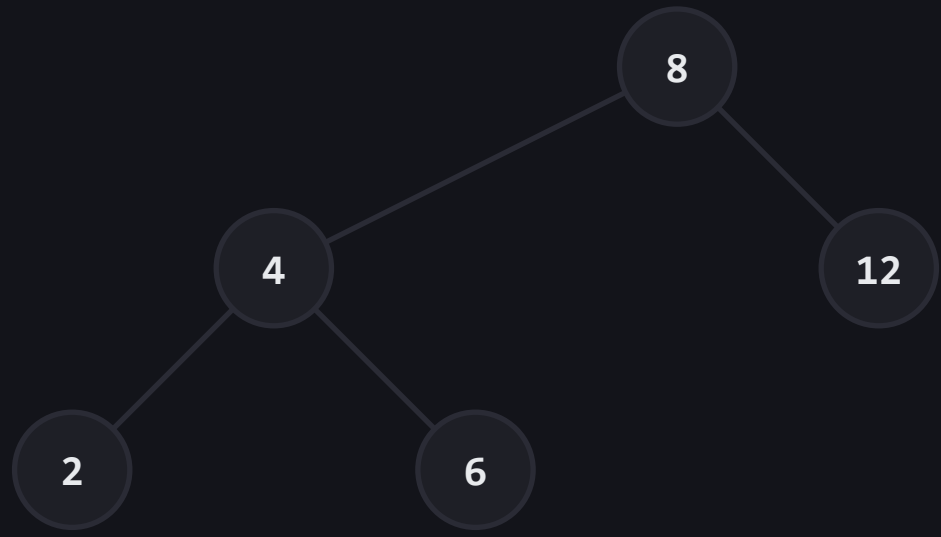
Esquerda, raiz, direita. Sempre produz os valores em ordem crescente.

Pré-ordem: raiz, esquerda, direita

Visita o nó atual antes de descer para os filhos.

Passo a passo

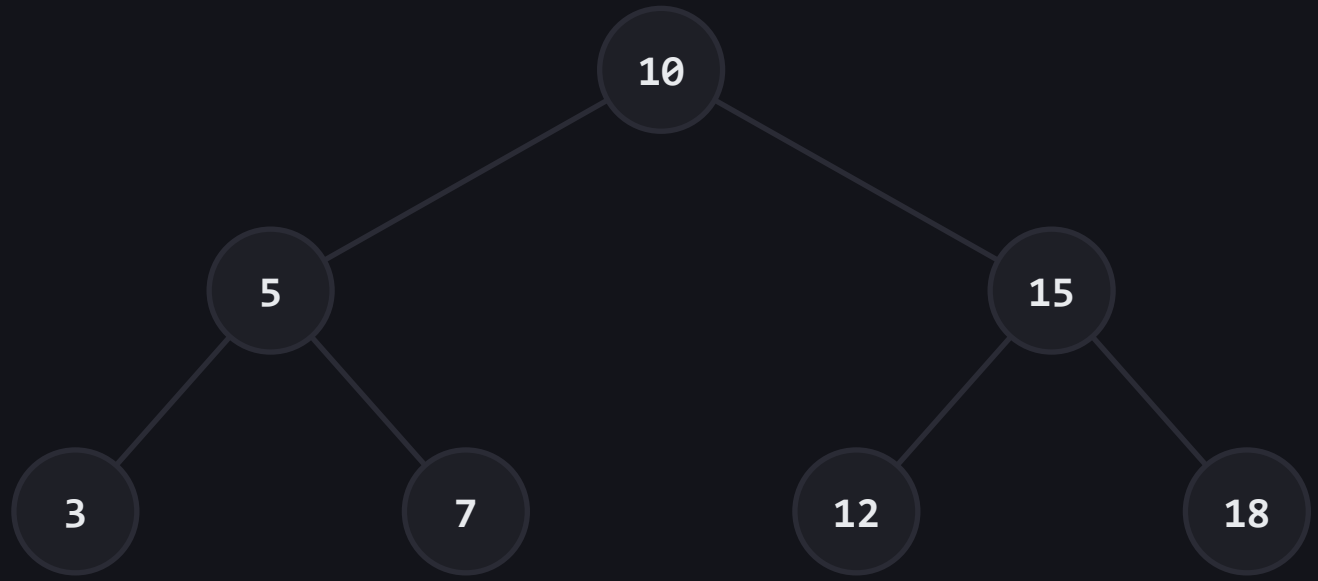
1. Visita 8, a raiz.
2. Desce à esquerda, visita 4.
3. Desce à esquerda, visita 2.
4. Volta, à direita de 4, visita 6.
5. Volta à raiz, à direita, visita 12.



8 → 4 → 2 → 6 → 12

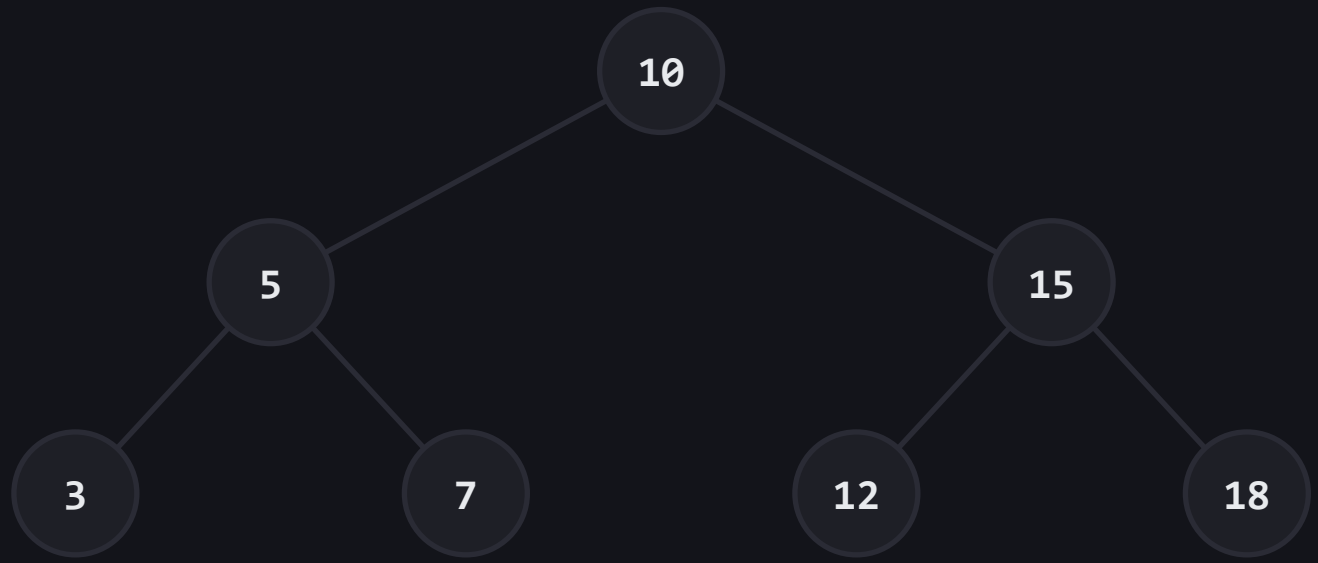
Pré-ordem

Escreva a sequência de visita do percurso pré-ordem para a árvore abaixo.



Pré-ordem: ____ , ____ , ____ , ____ , ____ , ____ , ____

Pré-ordem



10 → 5 → 3 → 7 → 15 → 12 → 18

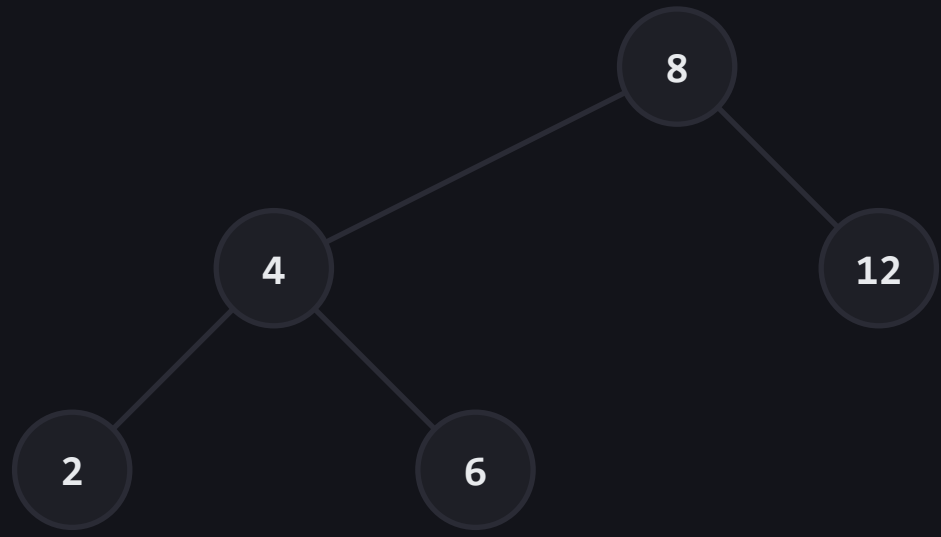
Raiz, esquerda, direita. A raiz é sempre o primeiro valor da sequência.

Pós-ordem: esquerda, direita, raiz

Visita o nó atual depois de processar os dois filhos.

Passo a passo

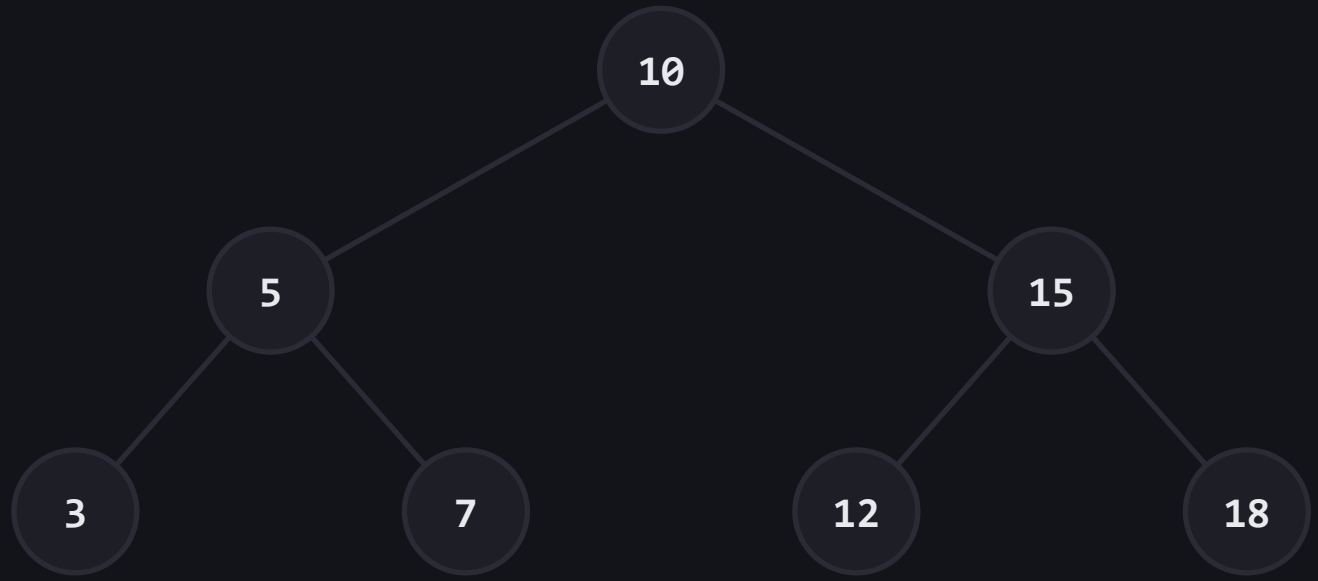
1. Desce à folha mais à esquerda, visita 2.
2. À direita de 4, visita 6.
3. Os filhos de 4 terminaram, visita 4.
4. À direita da raiz, visita 12.
5. Todos os filhos prontos, visita 8.



2 → 6 → 4 → 12 → 8

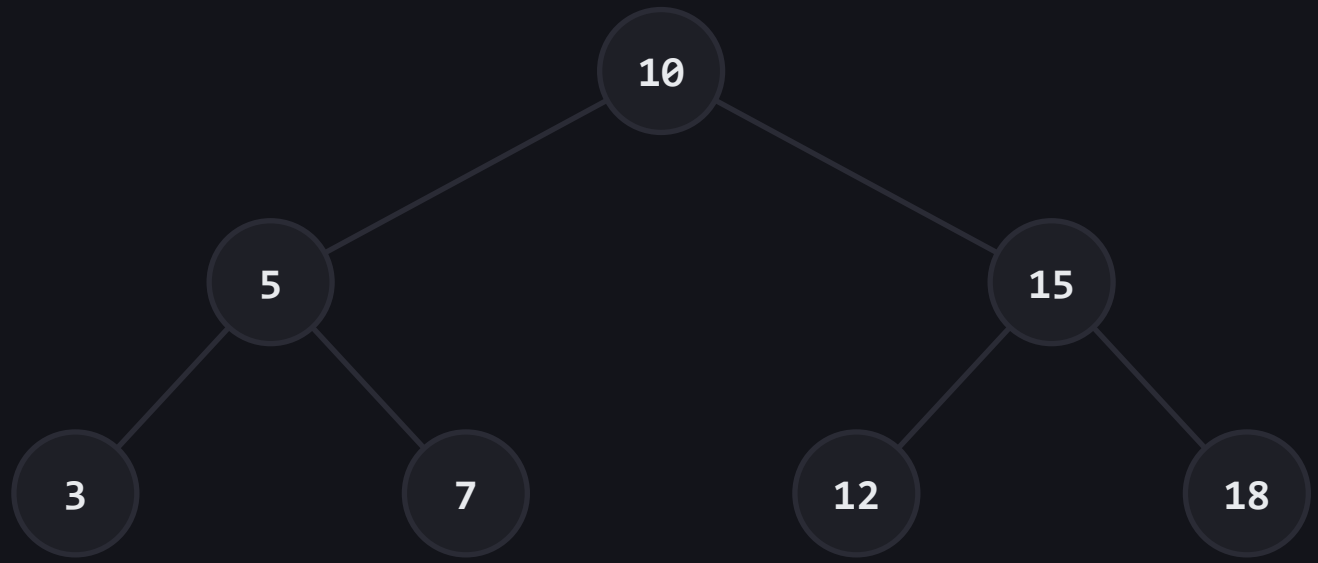
Pós-ordem

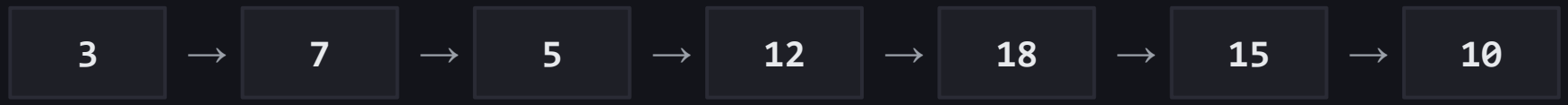
Escreva a sequência de visita do percurso pós-ordem para a árvore abaixo.



Pós-ordem: ____ , ____ , ____ , ____ , ____ , ____ , ____

Pós-ordem





Esquerda, direita, raiz. As folhas aparecem primeiro e a raiz por último.

Altura, profundidade e subárvore

Altura

Número de arestas no caminho mais longo da raiz até uma folha.

Uma árvore só com a raiz tem altura zero.

Profundidade

Número de arestas de um nó até a raiz.

A raiz tem profundidade zero, e um filho dela tem profundidade um.

Subárvore

Um nó e todos os seus descendentes.

A propriedade da BST vale dentro de cada subárvore.

A ordem da entrada muda a árvore

Os dois grupos abaixo têm exatamente os mesmos cinco valores,
e mudam apenas na ordem em que são inseridos.

Primeira árvore

Insira 1, 2, 3, 4, 5

Segunda árvore

Insira 3, 2, 4, 1, 5

Depois de desenhar as duas

Qual é a altura de cada uma?
Quantas comparações cada uma precisa para encontrar o valor 5?

As duas são BSTs válidas com os mesmos valores, mas não custam o mesmo.

Construir e modificar a árvore

Partindo de uma árvore vazia, execute as operações na ordem indicada e desenhe o estado da árvore depois de cada uma.

Operações

1. Inserir 15
2. Inserir 8
3. Inserir 20
4. Inserir 5
5. Inserir 12
6. Inserir 17
7. Buscar 12
8. Remover 8
9. Inserir 10
10. Buscar 10

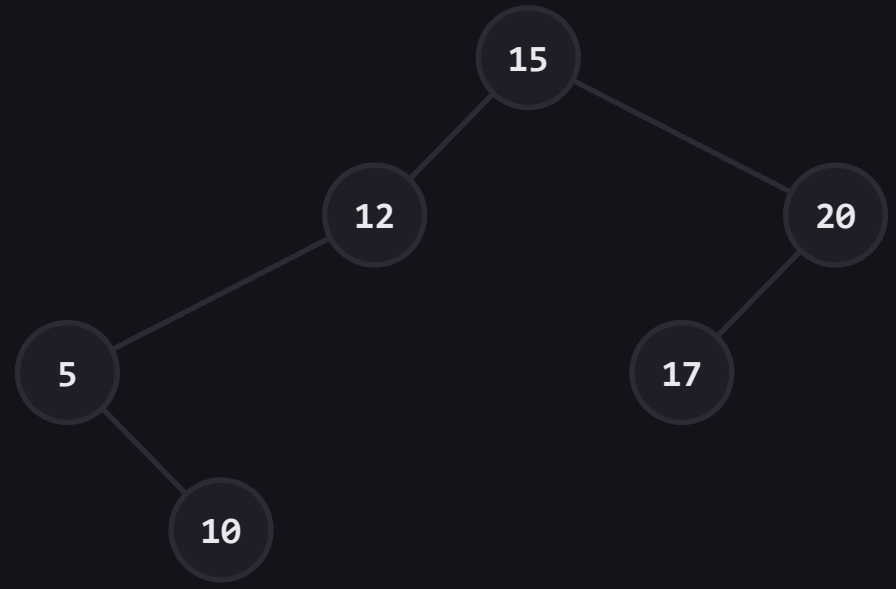
Depois de inserir os seis primeiros valores: qual nó é a raiz, qual é a altura da árvore e quais são as folhas?

Construir e modificar a árvore

Depois das seis inserções



Depois de remover 8 e inserir 10



Raiz

15, o primeiro valor inserido.

Altura

2 arestas até a folha mais distante.

Folhas

5, 12 e 17.

Remover 8, que tem dois filhos, substitui o valor pelo sucessor em-ordem, o 12.

Os três casos

Remover é a operação mais trabalhosa, porque depende de quantos filhos o nó tem.

Caso 1: nó folha

O nó não tem filhos.

Remove o nó e o pai passa a apontar para nullptr.

Caso 2: um filho

O nó tem um único filho.

Remove o nó e liga o filho diretamente ao pai.

Caso 3: dois filhos

O nó tem os dois filhos.

Substitui o valor pelo sucessor em-ordem, o menor da subárvore direita, e remove o sucessor.

A implementação da remoção fica para o próximo encontro.

A estrutura do nó

```
struct No {
    int valor;
    No* esq;
    No* dir;

    No(int novoValor) {
        valor = novoValor;
        esq = nullptr;
        dir = nullptr;
    }
};
```

Dois ponteiros

Cada nó guarda um valor e as ligações para os dois filhos.

A árvore inteira

É acessada apenas pelo ponteiro da raiz. Se ele for nullptr, a árvore está vazia.

O construtor já deixa os dois filhos vazios, então todo nó novo nasce como folha.

A inserção

```
No* inserir(No* raiz, int valor) {
    if (raiz == nullptr) {
        return new No(valor);
    }
    if (valor < raiz->valor) {
        raiz->esq = inserir(raiz->esq, valor);
    } else if (valor > raiz->valor) {
        raiz->dir = inserir(raiz->dir, valor);
    }
    return raiz;
}
```

O caso base

Chegou a uma posição vazia. É ali que o nó novo é criado.

O retorno

Cada chamada devolve a raiz da sua subárvore, e o pai a guarda de volta.

Valor igual ao do nó não entra nos dois if, então duplicatas são ignoradas.

Pré-ordem e em-ordem

```
void preOrdem(No* raiz) {
    if (raiz == nullptr) {
        return;
    }
    cout << raiz->valor << " ";
    preOrdem(raiz->esq);
    preOrdem(raiz->dir);
}
```

Saída: 8 4 2 6 12 10 14

```
void emOrdem(No* raiz) {
    if (raiz == nullptr) {
        return;
    }
    emOrdem(raiz->esq);
    cout << raiz->valor << " ";
    emOrdem(raiz->dir);
}
```

Saída: 2 4 6 8 10 12 14

A ordem das três linhas dentro da função define tudo. A única diferença entre os percursos é quando a raiz é visitada.

Pós-ordem

```
void posOrdem(No* raiz) {
    if (raiz == nullptr) {
        return;
    }
    posOrdem(raiz->esq);
    posOrdem(raiz->dir);
    cout << raiz->valor << " ";
}
```

Saída: 2 6 4 10 14 12 8

Liberar a memória

Libera os filhos antes do pai, então nunca sobra ponteiro solto.

Avaliar expressões

Em árvores de expressão, os operandos saem antes do operador.

Calcular tamanhos

Calcula o tamanho das subárvores antes de informar ao pai.

No repositório, entrega por pull request

A struct No vem pronta em todos os arquivos. O suporte diminui a cada exercício.

1. inserir

Escrever a inserção recursiva.

2. percursos

Escrever emOrdem, preOrdem e posOrdem.

3. pesquisa

Dizer se um valor existe na árvore.

4. contarNos

Contar quantos nós a árvore tem.

5. altura

Calcular a altura da árvore.

6. menorValor e maiorValor

Encontrar os extremos da árvore.

Cada aluno cria uma pasta com o próprio nome no repositório de entregas, faz commit e abre o pull request.

Cadastro de produtos

Uma loja precisa gerenciar produtos por código. Os códigos são inteiros únicos e o sistema deve manter sempre a ordem para facilitar a busca.

1. Cadastrar produto

Insere um código na árvore.

2. Buscar produto

Diz se o código existe na árvore.

3. Listar em ordem

Exibe os códigos em ordem crescente.

4. Total cadastrado

Quantos produtos estão cadastrados.

Teste com estes códigos, nesta ordem

50, 30, 70, 20, 40, 60, 80, 10, 35

FECHAMENTO

O que fica desta aula

A árvore binária de busca equilibra busca, inserção e remoção, e mantém os dados em ordem.

A propriedade é simples: à esquerda apenas menores, à direita apenas maiores.

Buscar e inserir seguem o mesmo caminho. A diferença é o que acontece ao chegar no vazio.

Os três percursos têm o mesmo código, só muda a posição da linha que imprime.

O percurso em-ordem devolve os valores ordenados sem nenhum algoritmo de ordenação.

No próximo encontro: a remoção completa e as árvores N-árias.